DESIGN OF ALGORITHMS

Complete Answer Book — All 8 Units

GTU Exam 2026 | Theory + Numerical + Algorithm Answers

Covers every Very Important and GTU Expected Question

Units	8
Questions Answered	80+
Topics	Theory, Numerical, Algorithms
Exam Year	2026

Unit 1: Introduction and Analysis of Algorithm

Topics Covered: *Algorithm basics, Time Complexity, Asymptotic Notations, Union-Find*

Q1. Define algorithm and explain properties of a good algorithm.

Definition

An **algorithm** is a finite, ordered set of well-defined instructions that takes some input and produces a desired output in a finite number of steps.

Properties of a Good Algorithm

- **Input:** Zero or more externally supplied inputs.
- **Output:** At least one output is produced.
- **Definiteness:** Each instruction is clear and unambiguous.
- **Finiteness:** The algorithm terminates after a finite number of steps.
- **Effectiveness:** Every step must be basic enough to be carried out.
- **Correctness:** Produces the correct output for all valid inputs.

Q2. Explain Best Case, Average Case and Worst Case analysis with example.

Complexity Cases

Best Case: Minimum time required for any input of size n. For Linear Search — O(1): element found at first position.

Average Case: Average time over all possible inputs of size n. For Linear Search — O(n/2) = O(n).

Worst Case: Maximum time required for any input of size n. For Linear Search — O(n): element at last position or absent.

```
Example – Linear Search: Best Case: O(1) key found at A[0] Average Case: O(n/2) key found near middle Worst Case: O(n) key at end or not present
```

Q3. Explain Big-O notation with examples.

Big-O Notation

Big-O describes the **upper bound** of an algorithm's running time. Formal definition: f(n) = O(g(n)) if there exist positive constants c and n0 such that f(n) <= c*g(n) for all n >= n0.

Common Examples

- O(1) — Constant time: Accessing array element A[i]
- O(log n) — Logarithmic: Binary Search
- O(n) — Linear: Linear Search
- O(n log n) — Merge Sort, Heap Sort
- O(n^2) — Quadratic: Bubble Sort, Selection Sort
- O(2^n) — Exponential: Recursive Fibonacci (naive)

Q4. Differentiate Big-O, Big-Theta and Big-Omega notation.

Notation	Meaning	Bound Type	Example
O(g(n))	f(n) <= c*g(n)	Upper Bound (Worst Case)	O(n^2) for Bubble Sort
Omega(g(n))	f(n) >= c*g(n)	Lower Bound (Best Case)	Omega(n log n) for comparison sort
Theta(g(n))	c1*g(n) <= f(n) <= c2*g(n)	Tight Bound (Exact)	Theta(n^2) for Selection Sort

Q5. Explain asymptotic analysis in detail.

Asymptotic Analysis

Asymptotic analysis evaluates algorithm performance as input size n grows toward infinity, ignoring constants and lower-order terms. Focus is on the growth rate.

Example: f(n) = 3n^2 + 5n + 10 is asymptotically O(n^2) since n^2 dominates. Allows comparison of algorithms independent of hardware or language.

Q6. Write Union and Find algorithms for disjoint set.

Disjoint Set — Union and Find

Find(x): Returns the representative (root) of the set containing x. Uses path compression for efficiency.

```
Find(x): if parent[x] != x: parent[x] = Find(parent[x]) // Path Compression return parent[x]
```

Union(x, y): Merges two sets. Uses union by rank to keep tree shallow.

```
Union(x, y): rootX = Find(x); rootY = Find(y) if rootX == rootY: return if rank[rootX] < rank[rootY]: parent[rootX] = rootY elif rank[rootX] > rank[rootY]: parent[rootY] = rootX else: parent[rootY] = rootX; rank[rootX]++
```

Time Complexity: O(alpha(n)) amortized — nearly constant (alpha = inverse Ackermann function).

Q7. Explain applications of disjoint set operations.

Applications of Disjoint Sets

- **Kruskal's MST Algorithm:** Detecting cycles when adding edges to MST.
- **Network Connectivity:** Checking if two nodes are in same connected component.
- **Image Processing:** Connected component labeling in binary images.
- **Social Networks:** Finding friend groups/communities.
- **Percolation problems** and dynamic connectivity queries.

Q8. What is time complexity and space complexity?

Time and Space Complexity

Time Complexity: Number of basic operations performed by an algorithm as a function of input size n. Expressed using Big-O notation (e.g., O(n log n)).

Space Complexity: Amount of memory used by an algorithm as a function of n. Includes input space + auxiliary/extra space.

Example: Merge Sort — Time: O(n log n) | Space: O(n) due to auxiliary array. Quick Sort — Time: O(n log n) avg | Space: O(log n) for recursion stack.

Q9. Compare iterative and recursive algorithms.

Aspect	Iterative	Recursive
Speed	Generally faster	Slower due to function call overhead
Space	O(1) extra space	O(n) stack space for recursion
Readability	Less elegant for complex problems	More readable and elegant
Risk	No stack overflow	Stack overflow on deep recursion
Example	Iterative Factorial	Recursive Fibonacci / Merge Sort

Unit 2: Analysis of Sorting Algorithms

Topics Covered: *Bubble, Selection, Insertion, Heap, Counting, Radix, Bucket Sort*

Q1. Explain Bubble Sort with example and complexity analysis.

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in wrong order. Largest elements bubble up to the end in each pass.

```
BubbleSort(A, n): for i = 0 to n-2: for j = 0 to n-2-i: if A[j] > A[j+1]: swap(A[j], A[j+1])
```

Example: A = [5, 3, 8, 1]

```
Pass 1: [3,5,1,8] Pass 2: [3,1,5,8] Pass 3: [1,3,5,8] Sorted!
```

Complexity: Best O(n) [already sorted, with flag] | Average O(n^2) | Worst O(n^2) | Space O(1) | Stable: Yes

Q2. Explain Selection Sort with suitable example.

Selection Sort

Finds minimum element from unsorted portion and swaps it to the beginning. Repeats for remaining elements.

```
SelectionSort(A, n): for i = 0 to n-2: min_idx = i for j = i+1 to n-1: if A[j] < A[min_idx]: min_idx = j swap(A[i], A[min_idx])
```

Example: [64, 25, 12, 22, 11]

```
Pass 1: min=11 at idx 4, swap -> [11, 25, 12, 22, 64] Pass 2: min=12 at idx 2, swap -> [11, 12, 25, 22, 64] Pass 3: min=22 at idx 3, swap -> [11, 12, 22, 25, 64] Pass 4: min=25 stays -> [11, 12, 22, 25, 64] Done!
```

Complexity: Best/Average/Worst O(n^2) | Space O(1) | Stable: No

Q3. Explain Heap Sort with algorithm and example.

Heap Sort

Uses a Max-Heap structure. Phase 1 builds max-heap from the array. Phase 2 repeatedly extracts the maximum (root) and heapifies.

```
Heapify(A, n, i): largest=i; l=2*i+1; r=2*i+2 if l<n and A[l]>A[largest]: largest=l if r<n and A[r]>A[largest]: largest=r if largest != i: swap(A[i], A[largest]) Heapify(A, n, largest) HeapSort(A, n): for i = n/2-1 down to 0: Heapify(A, n, i) // Build Max-Heap for i = n-1 down to 1: swap(A[0], A[i]) // Move root to end Heapify(A, i, 0) // Heapify reduced heap
```

Example: [4,10,3,5,1] -> Build Max-Heap: [10,5,3,4,1] -> Extract: [1,3,4,5,10]

Complexity: Best/Average/Worst O(n log n) | Space O(1) | Stable: No

Q4. Explain Counting Sort. Explain Radix Sort with example.

Counting Sort

Counts occurrences of each distinct element, builds prefix sums, then places elements in sorted positions. Not comparison-based.

```
CountingSort(A, n, k): // k = max value C[0..k] = 0 for i=0 to n-1: C[A[i]]++ for i=1 to k: C[i] += C[i-1] // prefix sum for i=n-1 to 0: B[C[A[i]]-1] = A[i]; C[A[i]]--
```

Advantages: O(n+k) time, stable. Disadvantage: O(k) space, only integers, poor when k >> n.

Radix Sort

Sorts digit by digit from least significant to most significant using stable sort (Counting Sort) at each digit.

```
A = [170, 45, 75, 90, 802, 24, 2, 66] After 1s place: [170, 90, 802, 2, 24, 45, 75, 66] After 10s place: [802, 2, 24, 45, 66, 170, 75, 90] After 100s place:[2, 24, 45, 66, 75, 90, 170, 802] Sorted!
```

Complexity: O(d*(n+k)) where d=number of digits, k=base (10)

Q5. Compare all sorting algorithms based on time and space complexity.

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)	Yes
Bucket Sort	O(n+k)	O(n+k)	O(n^2)	O(n)	Yes

Unit 3: Divide and Conquer

Topics Covered: *Binary Search, Merge Sort, Quick Sort, Matrix Multiplication, Recurrences*

Q1. Explain Divide and Conquer strategy with example.

Divide and Conquer

Divide and Conquer has three steps:

- **Divide:** Split the problem into smaller sub-problems of the same type.
- **Conquer:** Recursively solve sub-problems. Base cases solved directly.
- **Combine:** Merge solutions of sub-problems into the final solution.

Example — Merge Sort: Divide array into two halves, sort each recursively, merge the sorted halves. Recurrence: T(n) = 2T(n/2) + O(n) = O(n log n).

Q2. Explain Merge Sort with example and time complexity derivation.

Merge Sort

```
MergeSort(A, l, r): if l < r: mid = (l+r)/2 MergeSort(A, l, mid) MergeSort(A, mid+1, r) Merge(A, l, mid, r)
```

Example: A = [38, 27, 43, 3, 9, 82, 10]

```
Divide: [38,27,43,3] and [9,82,10] Sort: [3,27,38,43] and [9,10,82] Merge: [3, 9, 10, 27, 38, 43, 82] Sorted!
```

Time Complexity Derivation

```
T(n) = 2T(n/2) + cn [2 halves + merge O(n)] Expanding: T(n) = 4T(n/4) + 2cn = ... = n*T(1) + cn*log(n) = O(n log n)
```

Complexity: O(n log n) all cases | Space O(n) | Stable: Yes

Q3. Explain Quick Sort with example.

Quick Sort

Selects a pivot, partitions array so elements less than pivot go left and greater go right, then recursively sorts both partitions.

```
QuickSort(A, low, high): if low < high: pi = Partition(A, low, high) QuickSort(A, low, pi-1) QuickSort(A, pi+1, high) Partition(A, low, high): pivot = A[high]; i = low-1 for j = low to high-1: if A[j] <= pivot: i++; swap(A[i], A[j]) swap(A[i+1], A[high]) return i+1
```

Example: A=[10,7,8,9,1,5], pivot=5

```
Partition: [1,5,8,9,7,10] pivot 5 placed at index 1 Recurse on [1] and [8,9,7,10] Final: [1,5,7,8,9,10]
```

Complexity: Best/Average O(n log n) | Worst O(n^2) (sorted input) | Space O(log n)

Q4. Compare Merge Sort and Quick Sort.

Aspect	Merge Sort	Quick Sort
Worst Case	O(n log n)	O(n^2)
Average Case	O(n log n)	O(n log n)
Space	O(n) auxiliary	O(log n) in-place
Stable	Yes	No
Cache Performance	Poor (non-sequential access)	Better (sequential access)

Best For	Linked lists, external sorting	Arrays, general-purpose sorting

Q5. Explain Binary Search algorithm and complexity.

Binary Search

Works on sorted arrays. Compares target with middle element; eliminates half the search space each iteration.

```
BinarySearch(A, low, high, key): while low <= high: mid = (low+high)/2 if A[mid] == key: return mid if A[mid] < key: low = mid+1 else: high = mid-1 return -1
```

Complexity: Best O(1) | Average/Worst O(log n) | Space O(1) iterative

Q6. Explain Matrix Multiplication using Divide and Conquer (Strassen's).

Strassen's Matrix Multiplication

Standard n x n matrix multiplication needs O(n^3). Strassen divides each matrix into four n/2 x n/2 sub-matrices and uses only 7 multiplications (instead of 8):

```
M1=(A11+A22)(B11+B22) M2=(A21+A22)*B11 M3=A11*(B12-B22) M4=A22*(B21-B11) M5=(A11+A12)*B22
M6=(A21-A11)*(B11+B12) M7=(A12-A22)*(B21+B22) C11=M1+M4-M5+M7 C12=M3+M5 C21=M2+M4
C22=M1-M2+M3+M6
```

Complexity: T(n) = 7T(n/2) + O(n^2) => O(n^2.807) by Master Theorem (better than O(n^3)).

Unit 4: Dynamic Programming

Topics Covered: *Principle of Optimality, 0/1 Knapsack, LCS, Matrix Chain Multiplication*

Q1. Define Dynamic Programming and explain Principle of Optimality.

Dynamic Programming

Dynamic Programming (DP) is an algorithm design technique that solves complex problems by breaking them into overlapping sub-problems, solving each sub-problem only once, and storing the results in a table (memoization or tabulation).

Principle of Optimality (Bellman)

An optimal solution to a problem contains within it optimal solutions to its sub-problems. If overall path A to C through B is optimal, then path A to B must also be optimal.

Characteristics of DP

- **Optimal Substructure:** Optimal solution built from optimal sub-solutions.
- **Overlapping Sub-problems:** Same sub-problems solved multiple times (unlike D&C).
- **Memoization / Tabulation:** Store results to avoid recomputation.

Q2. Explain 0/1 Knapsack Problem using Dynamic Programming.

0/1 Knapsack — DP

Given n items with weights w[] and values v[], and knapsack capacity W — maximize total value. Each item is taken (1) or not (0).

```
K[i][w] = max value using first i items with capacity w Recurrence: K[i][w] = K[i-1][w] if w[i] > w K[i][w] = max(K[i-1][w], v[i]+K[i-1][w-w[i]]) otherwise
```

Example: n=3, W=5, weights=[2,3,4], values=[3,4,5]

```
w=0 1 2 3 4 5 i=0: 0 0 0 0 0 0 i=1: 0 0 3 3 3 3 (w=2, v=3) i=2: 0 0 3 4 4 7 (w=3, v=4) i=3: 0 0 3 4 5 7 (w=4, v=5) Answer = K[3][5] = 7 (take items 1 and 2)
```

Complexity: Time O(n*W) | Space O(n*W)

Q3. Explain Longest Common Subsequence (LCS) with example.

LCS — Dynamic Programming

Finds the longest subsequence common to two sequences. A subsequence preserves relative order but need not be contiguous.

```
c[i][j] = 0 if i=0 or j=0 c[i][j] = c[i-1][j-1]+1 if X[i] == Y[j] c[i][j] = max(c[i-1][j], c[i][j-1]) otherwise
```

Example: X = "ABCBDAB", Y = "BDCAB"

```
'' B D C A B '': 0 0 0 0 0 0 A: 0 0 0 0 1 1 B: 0 1 1 1 1 2 C: 0 1 1 2 2 2 B: 0 1 1 2 2 3 D: 0 1 2 2 2 3 A: 0 1 2 2 3 3 B: 0 1 2 2 3 4 LCS Length = 4 (e.g., BCAB or BDAB)
```

Complexity: Time O(m*n) | Space O(m*n)

Q4. Explain Matrix Chain Multiplication.

Matrix Chain Multiplication — DP

Find the optimal parenthesization of matrix chain A1*A2*...*An that minimizes total scalar multiplications.

```
m[i][j] = min multiplications to compute Ai..Aj m[i][i] = 0 m[i][j] = min over i<=k<j of: m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]
```

Example: A(10x30), B(30x5), C(5x60) => p = [10,30,5,60]

```
m[1][2] = 10*30*5 = 1500 m[2][3] = 30*5*60 = 9000 m[1][3] = min( m[1][1]+m[2][3]+10*30*60 = 0+9000+18000 = 27000, m[1][2]+m[3][3]+10*5*60 = 1500+0+3000 = 4500 ) = 4500 Optimal parenthesization: A*(B*C)
```

Complexity: Time O(n^3) | Space O(n^2)

Q5. Compare Divide and Conquer with Dynamic Programming.

Aspect	Divide and Conquer	Dynamic Programming
Sub-problems	Independent	Overlapping
Storage	No memoization needed	Uses memoization/DP table
Approach	Top-down (recursive)	Bottom-up or Top-down
Recomputation	Sub-problems recomputed	Each solved only once
Examples	Merge Sort, Quick Sort	Knapsack, LCS, Floyd-Warshall

Unit 5: Greedy Algorithm

Topics Covered: *Greedy Strategy, Activity Selection, Fractional Knapsack, Job Scheduling*

Q1. Explain Greedy Method with characteristics.

Greedy Algorithm

A **Greedy Algorithm** builds a solution step by step, always making the locally optimal choice at each step, hoping to reach a globally optimal solution.

Characteristics

- **Greedy Choice Property:** A globally optimal solution can be reached by making locally optimal choices.
- **Optimal Substructure:** Optimal solution contains optimal solutions to sub-problems.
- **Irrevocability:** Once a choice is made, it is never reconsidered.
- **Efficiency:** Usually O(n log n) due to initial sorting step.

Q2. Explain Activity Selection Problem.

Activity Selection Problem

Given n activities with start s[] and finish f[] times, select maximum number of non-overlapping activities.

Greedy Rule: Always pick the activity with the earliest finish time that does not conflict with previously selected activities.

```
ActivitySelection(s[], f[], n): Sort activities by finish time selected = {A[0]}; k = 0 for i = 1 to n-1: if s[i] >= f[k]: // no overlap selected.add(A[i]); k = i return selected
```

Example: Finish times sorted: 4,5,6,7,9,9,10,11,12,14,16

Select: A1(1-4), A4(5-7), A8(8-11), A11(12-16) => 4 activities (maximum possible)

Complexity: O(n log n) for sorting + O(n) for selection

Q3. Explain Fractional Knapsack Problem using Greedy Method.

Fractional Knapsack

Items can be taken in fractions. Greedy strategy: take items with highest value/weight ratio first.

```
FractionalKnapsack(items, W): Compute ratio = value/weight for each item Sort by ratio descending totalValue = 0; remaining = W for each item in sorted order: if item.weight <= remaining: take whole item remaining -= item.weight else: fraction = remaining / item.weight totalValue += fraction * item.value break return totalValue
```

Example: W=50, Items(weight,value): (10,60),(20,100),(30,120)

```
Ratios: 60/10=6, 100/20=5, 120/30=4 Take item1 (wt=10): value=60, remaining=40 Take item2 (wt=20): value=160, remaining=20 Take 20/30 of item3: value=160+80=240 Max Value = 240
```

Complexity: O(n log n) | Greedy gives optimal for fractional, NOT for 0/1 Knapsack.

Q4. Differentiate Greedy Method and Dynamic Programming.

Aspect	Greedy	Dynamic Programming
Choice	Local optimum at each step	Considers all sub-problem combinations
Revision	Never revises choices	Not applicable (uses table)
Optimality	Not always optimal	Always finds optimal solution
Efficiency	Usually faster O(n log n)	O(n^2) or O(n^3) often
Use Case	Fractional Knapsack, MST, Activity Selection	0/1 Knapsack, LCS, MCM

Unit 6: Graph Algorithms

Topics Covered: *DFS, BFS, Prim's, Kruskal's MST, Dijkstra's, Topological Sort*

Q1. Explain BFS traversal with example.

Breadth-First Search (BFS)

BFS explores all neighbors at current depth before going deeper. Uses a **Queue**. Produces shortest path in unweighted graphs.

```
BFS(G, s): Mark s as visited; enqueue(Q, s) while Q not empty: u = dequeue(Q) for each neighbor v of u: if v not visited: mark v visited; enqueue(Q, v)
```

Example: Graph edges: 0-1, 0-2, 1-3, 2-3, 3-4. BFS from 0:

```
Queue: [0] -> visit 0 Queue: [1,2] -> visit 1, visit 2 Queue: [3] -> visit 3 Queue: [4] -> visit 4 BFS Order: 0 -> 1 -> 2 -> 3 -> 4
```

Complexity: O(V+E) adjacency list | O(V^2) adjacency matrix

Q2. Explain DFS traversal with example.

Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking. Uses a **Stack** or recursion.

```
DFS(G, u): Mark u as visited for each neighbor v of u: if v not visited: DFS(G, v)
```

Example (same graph), DFS from 0:

```
Visit 0 -> Visit 1 -> Visit 3 -> Visit 2 -> Visit 4 DFS Order: 0 -> 1 -> 3 -> 2 -> 4
```

Complexity: O(V+E) | Applications: Cycle detection, Topological sort, SCC

Q3. Differentiate BFS and DFS.

Aspect	BFS	DFS
Data Structure	Queue (FIFO)	Stack / Recursion (LIFO)
Traversal Order	Level by level	Branch by branch (deep first)
Shortest Path	Yes (unweighted graphs)	No
Space Usage	O(V) — problematic for wide graphs	O(V) — problematic for deep graphs
Applications	Shortest path, Web crawling, Social networks	Topological sort, Cycle detection, SCC

Q4. Explain Prim's Algorithm with example.

Prim's Algorithm — Minimum Spanning Tree

Greedy MST algorithm. Starts from any vertex; greedily adds the minimum-weight edge connecting the MST set to a new vertex.

```
Prim(G, s): key[v] = INF for all v; key[s] = 0 parent[v] = -1 Add all vertices to priority queue Q while Q not empty: u = extractMin(Q) for each neighbor v of u: if v in Q and w(u,v) < key[v]: parent[v] = u; key[v] = w(u,v)
```

Example: Edges: A-B(2), A-C(3), B-C(1), B-D(4), C-D(5)

```
Start A: pick B(2)->A-B, then C(1)->B-C, then D(4)->B-D MST edges: A-B(2), B-C(1), B-D(4) Total cost = 7
```

Complexity: O(V^2) with matrix | O(E log V) with min-heap

Q5. Explain Kruskal's Algorithm with example.

Kruskal's Algorithm — MST

Sorts all edges by weight; adds edge if it does not form a cycle (uses Union-Find for cycle detection).

```
Kruskal(G): Sort all edges by weight (ascending) Initialize Union-Find for all vertices MST = {}
for each edge (u,v,w) in sorted order: if Find(u) != Find(v): // no cycle MST.add(u,v,w)
Union(u, v)
```

Example (same graph): Sorted: B-C(1), A-B(2), A-C(3), B-D(4), C-D(5)

```
Add B-C(1): no cycle -> MST={B-C} Add A-B(2): no cycle -> MST={B-C, A-B} Add A-C(3): CYCLE ->
skip Add B-D(4): no cycle -> MST={B-C, A-B, B-D} Done! Total cost = 1+2+4 = 7
```

Complexity: O(E log E) or O(E log V)

Q6. Explain Dijkstra's Single Source Shortest Path Algorithm.

Dijkstra's Algorithm

Finds shortest paths from source s to all vertices in a weighted graph with non-negative weights only.

```
Dijkstra(G, s): dist[v] = INF for all v; dist[s] = 0 Priority Queue Q with all vertices while Q
not empty: u = extractMin(Q) for each neighbor v of u: if dist[u] + w(u,v) < dist[v]: dist[v] =
dist[u] + w(u,v) decreaseKey(Q, v, dist[v])
```

Complexity: O(V^2) with arrays | O((V+E) log V) with binary heap

Q7. Explain Topological Sorting.

Topological Sort

Linear ordering of vertices in a DAG such that for every directed edge (u,v), u appears before v. Used for task scheduling, build systems, course prerequisites.

```
DFS-based: DFS on all vertices; push to stack when vertex finishes Pop stack = topological order
Kahn's Algorithm (BFS): Compute in-degree of all vertices Enqueue all vertices with in-degree 0
While queue not empty: dequeue u; add to result For each neighbor v: in-degree[v]-- If
in-degree[v]==0: enqueue v
```

Complexity: O(V+E) | Only valid for DAGs (no cycles)

Unit 7: Backtracking and Branch and Bound

Topics Covered: *N-Queen, Sum of Subsets, Graph Coloring, Hamiltonian Cycle, TSP*

Q1. Explain Backtracking method with example.

Backtracking

Backtracking builds a solution incrementally and abandons (backtracks) each partial candidate as soon as it violates a constraint. Explores the state space tree using DFS.

```
Backtrack(candidate): if isComplete(candidate): output solution; return for each option in extensions(candidate): if isValid(candidate + option): add option to candidate Backtrack(candidate) remove option // backtrack
```

Examples: N-Queen, Sudoku, Subset Sum, Graph Coloring, Hamiltonian Cycle.

Q2. Explain N-Queen Problem using Backtracking. Solve 4-Queen.

N-Queen / 4-Queen Problem

Place N queens on N x N chessboard such that no two queens share the same row, column, or diagonal.

```
isSafe(board, row, col, n): Check all rows above in same column Check upper-left diagonal Check upper-right diagonal Return true if no conflict SolveNQueen(board, row, n): if row == n: print solution; return for col = 0 to n-1: if isSafe(board, row, col, n): board[row][col] = 1 SolveNQueen(board, row+1, n) board[row][col] = 0 // backtrack
```

4-Queen Solutions

```
Solution 1: Solution 2: . Q . . . . Q . . . . Q Q . . . Q . . . . . . . Q . . Q . . Q . . (Q=queen placed, .=empty)
```

State space tree: 4 choices per row, prune immediately when conflict detected.

Q3. Explain Branch and Bound method. Explain TSP using Branch and Bound.

Branch and Bound

Systematically enumerates candidate solutions using a state space tree. Uses a **bounding function** to eliminate subtrees that cannot produce better solutions than current best.

Types

- **FIFO B&B:** Uses Queue — explores level by level (BFS order).
- **LIFO B&B:** Uses Stack — explores depth-first.
- **LC B&B (Least Cost):** Uses Priority Queue — always expands most promising node.

TSP using Branch and Bound

Traveling Salesman Problem: Find minimum cost Hamiltonian cycle visiting all cities exactly once.

```
Algorithm: 1. Start with root (no city visited) 2. Compute lower bound for each partial path Lower bound = sum of (min 2 edges per city) / 2 3. Expand node with minimum lower bound (LC-B&B) 4. Prune if current bound >= best known solution 5. When all cities visited, update best if improved Example 4-city cost matrix: A B C D A [ 0 10 15 20 ] B [10 0 35 25 ] C [15 35 0 30 ] D [20 25 30 0 ] Optimal tour: A->B->D->C->A = 10+25+30+15 = 80
```

Q4. Compare Backtracking and Branch and Bound.

Aspect	Backtracking	Branch and Bound
Goal	Find feasible/all solutions	Find optimal solution
Bounding	No cost bound	Uses cost bound function
Traversal	Always DFS	BFS / LC / FIFO / LIFO

Pruning	Prune infeasible states	Prune non-optimal subtrees
Problem Type	Decision / enumeration problems	Optimization problems
Examples	N-Queen, Sudoku, Graph Coloring	TSP, Knapsack (optimal)

Unit 8: NP Completeness

Topics Covered: *Class P, NP, NP-Hard, NP-Complete, Polynomial Reduction, SAT*

Q1. Define class P and NP.

Class P

P (Polynomial Time) is the class of decision problems solvable in polynomial time O(n^k) by a deterministic Turing machine. These are considered efficiently solvable.

Examples: Sorting, Shortest Path, MST, Binary Search, Matrix Multiplication.

Class NP (Non-deterministic Polynomial)

NP is the class of decision problems whose solutions can be **verified** in polynomial time by a deterministic Turing machine (even if finding the solution is hard).

Every problem in P is also in NP (P is a subset of NP). The key open question: is P = NP?

Examples: SAT, Hamiltonian Cycle, Subset Sum, 0/1 Knapsack (decision version).

Q2. Differentiate NP-Hard and NP-Complete problems.

NP-Hard

A problem H is NP-Hard if every problem in NP can be polynomial-time reduced to H. NP-Hard problems are at least as hard as any NP problem, but may NOT be in NP themselves (solution may not be verifiable in polynomial time).

Examples: TSP (optimization), Halting Problem, General Graph Coloring.

NP-Complete

A problem C is NP-Complete if: (1) C is in NP (solution verifiable in poly time), AND (2) C is NP-Hard (every NP problem reduces to C in poly time). These are the hardest problems in NP.

Examples: SAT, 3-SAT, Vertex Cover, Clique, Hamiltonian Cycle, TSP (decision).

Q3. Explain Polynomial Reduction and Satisfiability Problem (SAT).

Polynomial Reduction

Problem A is polynomial-time reducible to problem B (A reduces_to B) if any instance of A can be transformed into an instance of B in polynomial time, such that YES-instance of A maps to YES-instance of B.

Used to prove NP-Completeness: to show X is NP-Complete, show (1) X is in NP, and (2) reduce a known NP-Complete problem to X in polynomial time.

SAT — Satisfiability Problem

Given a Boolean formula in CNF (Conjunctive Normal Form), is there a variable assignment that makes it TRUE?

```
Example: (x1 OR x2) AND (NOT x1 OR x3) AND (x2 OR NOT x3) Assignment: x1=T, x2=T, x3=T (T OR T) AND (F OR T) AND (T OR F) = T AND T AND T = TRUE Satisfiable!
```

SAT was the first problem proved NP-Complete (Cook-Levin Theorem, 1971). All subsequent NP-Complete proofs used reduction from SAT or its variants.

Q4. Compare P, NP, NP-Hard and NP-Complete.

Class	Solvable in Poly Time?	Verifiable in Poly Time?	Examples
P	Yes	Yes	Sorting, Binary Search, Shortest Path, MST
NP	Unknown (maybe)	Yes	SAT, Hamiltonian Cycle, Subset Sum

NP-Hard	Unknown	Not necessarily	TSP (optimization), Halting Problem
NP-Complete	Unknown (if yes: P=NP)	Yes	3-SAT, Vertex Cover, Clique, TSP (decision)

Relationship: P is subset of NP is subset of NP-Hard. NP-Complete = NP intersect NP-Hard. The P vs NP question (whether P=NP) is the most famous unsolved problem in theoretical computer science.

Q5. Explain importance of NP-Completeness.

Importance of NP-Completeness

- Identifies computationally intractable problems — no efficient algorithm likely exists.
- Guides use of approximation algorithms, heuristics, and randomized approaches instead.
- Cryptographic security relies on NP-hard assumptions (integer factorization, discrete log).
- Helps classify new problems: reduce from known NP-Complete problem to prove hardness.
- Central framework of theoretical computer science and complexity theory.
- If any NP-Complete problem is solved in poly time, ALL NP problems become tractable (P=NP).

100% IMPORTANT — Must Prepare List with Key Points

No.	Topic	Key Points to Remember
1	**Merge Sort**	O(n log n) all cases | Stable | Space O(n) | Recurrence: T(n)=2T(n/2)+cn
2	**Quick Sort**	Avg O(n log n), Worst O(n^2) | In-place | Pivot partitioning | Median-of-three for better performance
3	**DP – 0/1 Knapsack**	K[i][w] table | Recurrence: max(K[i-1][w], vi+K[i-1][w-wi]) | O(nW) time and space
4	**LCS**	c[i][j] table | Match: diagonal+1 | Mismatch: max(left,up) | O(mn) | Backtrack for actual sequence
5	**Prim's Algorithm**	Greedy MST | Start any vertex | Min edge to new vertex | O(E log V) with heap
6	**Kruskal's Algorithm**	Greedy MST | Sort edges | Union-Find for cycle check | O(E log E)
7	**BFS and DFS**	BFS: Queue, shortest path O(V+E) | DFS: Stack/recursion, cycle detect O(V+E)
8	**N-Queen Problem**	Backtracking | Place row by row | Check col+both diagonals | Backtrack on conflict
9	**Branch and Bound – TSP**	LC-B&B; | Lower bound = min edges | Priority queue | Prune if bound >= best
10	**NP-Complete vs NP-Hard**	NP-Complete: in NP AND NP-Hard | NP-Hard: may not be in NP | SAT is first NP-Complete

Best of Luck for Your GTU Examination 2026!

Study Smart. Revise Well. You've Got This!